

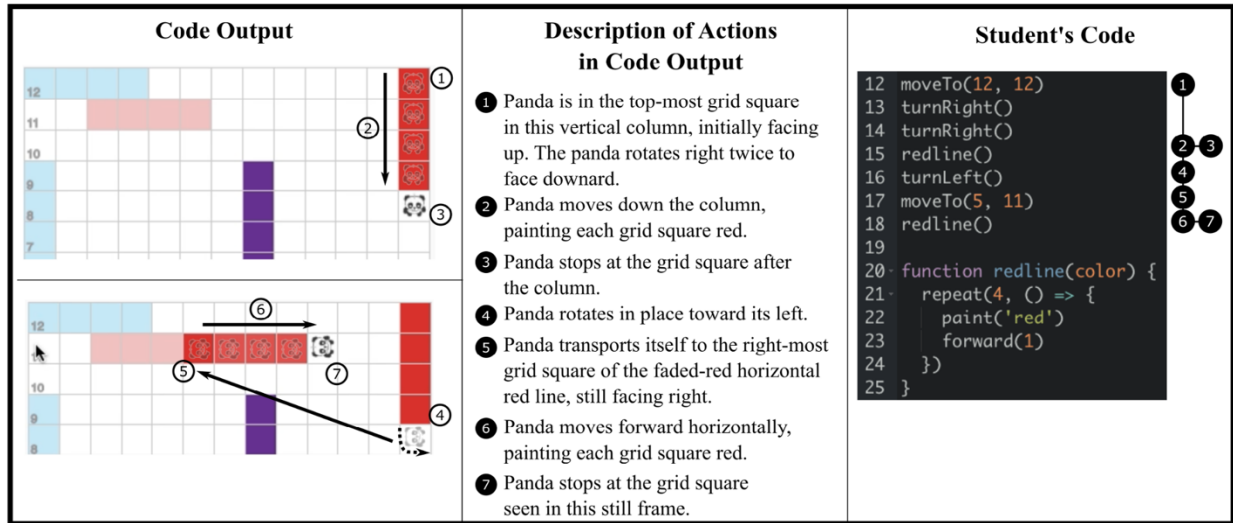
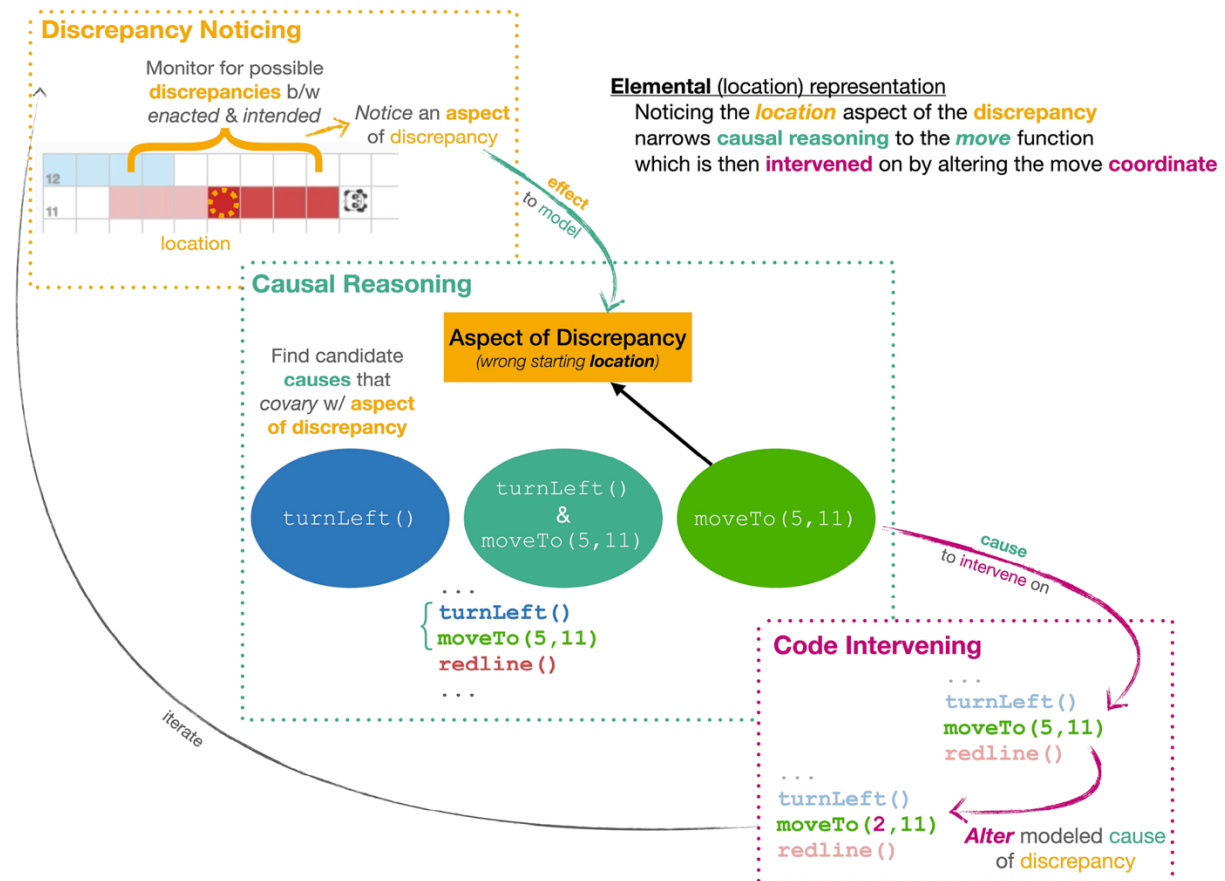


Fig. 1. Kay's code (right), code output (left), and description of output (middle). Numbers 1–7 match code with output.



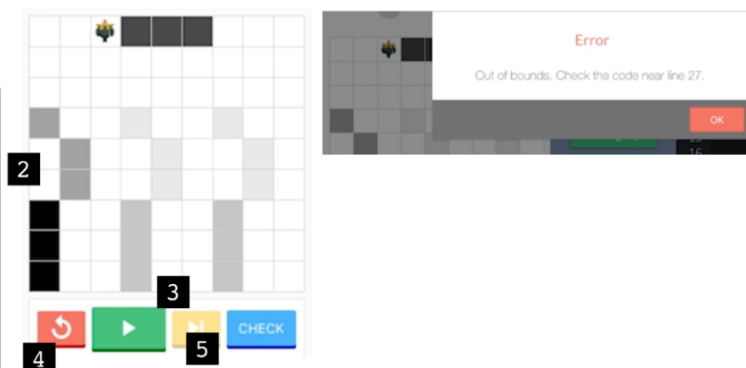
Ana's code at the start of this transcript. Everything except for the `drawJaggedLine()` function definition was provided as starter code.

```

1 //DEFINE the function drawJaggedLine().
2 //The function moveToNextLine() is
3 //already defined for you.
4
5
6 drawJaggedLine()
7
8 moveToNextLine()
9
10 drawJaggedLine()
11
12 moveToNextLine()
13
14 drawJaggedLine()
15
16
17
18 //PixelBot should face the same way at
19 //the start and end of this function.
20 function drawJaggedLine() {
21   paint('black')
22   forward(1)
23   paint('black')
24   forward(1)
25   paint('black')
26   forward(1)
27   turnRight()
28   forward(1)
29   paint('grey')
30   turnLeft()
31   forward(1)
32   paint('grey')
33   forward(1)
34   turnLeft()
35   forward(1)
36   paint('grey')
37 }
38
39
40 //This function moves from the end of
41 //one jagged line to the start of the
42 //next. If the PixelBot is facing in
43 //the wrong direction, this won't work
44 //correctly.
45 function moveToNextLine() {
46   turnRight()
47   turnRight()
48   forward(5)
49   turnLeft()
50   forward(3)
51   turnLeft()
52 }

```

The Toucan's location right before it goes out of bounds and generates an error message.



[Ana and their instructor have been discussing the bug. Mel, seated next to Ana proposes that Ana needs to fix the `moveToNextLine()` function. The instructor continues the discussion as follows.]

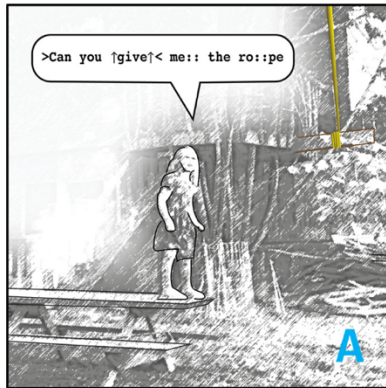
01 INS: So sometimes? (.) if: the code is already
 02 there for you? (.)
 03 [some- >sometimes you won't be able to change
 04 this:: so you might have to make
 05 1 [((moves right hand on top of laptop screen,
 06 covering entire `moveToNextLine` function))

07 [yu::r co::de? (1.2)
 08 2 [((right hand drifts over to far left of code
 09 output, forms a bracket between thumb and
 10 pointer finger, and hovers above screen))

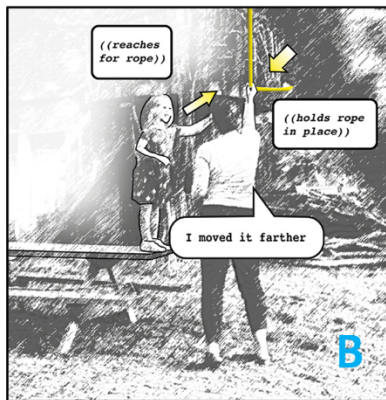
11 [wo:rk?
 12 [((bracket gesture twists to the right))
 13 for a particular kind of way::,
 14 [so that it goes to the
 15 3 [((right finger points to bottom center of
 16 code output))

17 right location (0.8)
 18 So:: (1.2) [let's reset our
 19 4 [((points at red reset button))

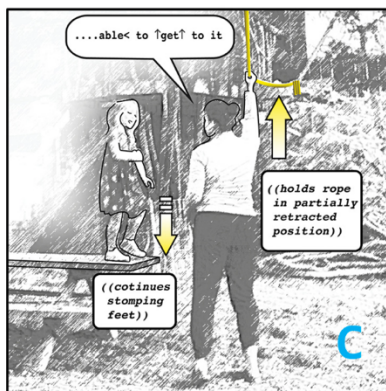
20 pixel bot (.)
 21 [and let's go one line at a time and let's
 22 figure out like how we can (.) maybe change
 23 our:: uh jagged line function? (0.4) to see
 24 um:: (1.2) to get to the right location
 25 ANA: 5 [((clicks reset and starts clicking stepper
 26 button repeatedly to move line by line
 27 through code))



01 Rae: [((climbs up on picnic table, approaches rope, walks to end of
02 table top))]
03 Mom: [((pulls back swing while looking over her shoulder toward
04 Rae, runs through foreground to push swing))]
05 Rae: [((stops at end of table, turns toward Mom))]
06 Rae: [>Can you ↑give↑< me::]
07 Rae: [the ro::pe. (0.5) can you give me the ro::pe.]
08 [((steps toward edge of table, rocks
09 side-to-side, holding hands down by waist))]
10 Mom: Oh >I ↑kinda↑ moved< th-
11 [the table >so it wouldn't be< so close]
12 [((walks back toward Rae))]
13 Rae: [((tip-toes forward until feet hang over edge of table))]



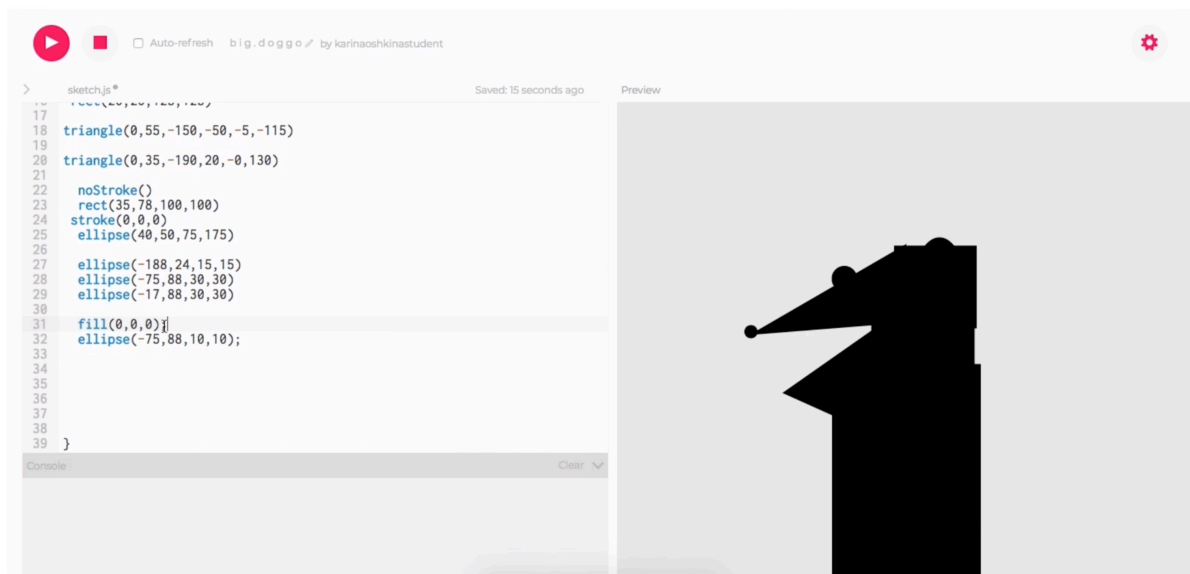
14 Mom: [((reaches toward rope, looks up at rope, back to Rae, up at
15 rope))]
16 U::m [(1.5)]
17 [((grabs the rope, pulls it toward Rae))]
18 Rae: [((reaches for rope))]
19 Mom: [I moved it farther]
20 [((holds rope in place))]
21 Mom: [away]
22 Rae: [((pulls hand back down quickly))]
23 Mom: [((partially retracts her pull of the rope))]
24 [so >you wouldn't be]
25 Rae: [HM ↓m::↓]
26 [((stomps feet on table, while stepping back from edge))]



27 Mom: [able to< ↑get↑ to it]
28 [((holds rope in partially retracted position))]
29 Rae: [((continues stomping feet))]
30 Rae: [But I WA::Nto]
31 [((returns to edge of table, toes over edge,
32 spreads arms and tilts head))]
33 Mom: >Okay< I'll move it ↑back↑ over (.3)
34 Mom: [just hop off]
35 Rae: [((turns toward side of table))]
36 Mom: for (.3) (for a sec)()
37 Rae: [((jumps down with two hops))]
38 [whee yay ↑YAY↑]

((TEO adds code to Flo's program to slow down its frame rate. This makes it so that Flo's code, when it is run, fills in the image slowly so that Flo can see how the design is drawn by the program and perhaps better understand what is causing the problem))

01 TEO: So the first time it goes to draw your do::g (1.2) it/s all white(.)
 02 like how you had it right,
 03 FLO: Mhm
 04 TEO: And then the SECOND time (.) and then forever after that it's all::: (0.4
 05 filled (.) black.
 06 Flo: ((nervous squeaks))
 07 TEO: So (2.0) >where do you think you should start< (0.4) so that's- that's
 08 what's happening. (0.5) so what are your thoughts about that. (0.6) what
 09 are you thinking.
 10 FLO: ((deep inhale)) ↑OH BOY↑
 11 TEO: ((light laugh)) so what did you::: change first (1.2)
 12 [to:: make that happen
 13 FLO: [The fil::l
 14 *[((clicks next to fill function on line 31))*



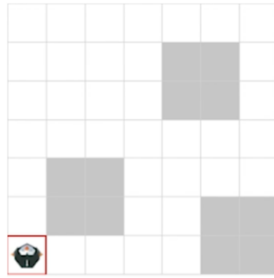
15 TEO: Okay. (1.6) >So let's experiment with that<
 16 FLO: *((deletes fill(0,0,0) above last ellipse function and runs code))*
 17 *((dog design now appears all white again))*
 18 TEO: Mokay interesti::ng
 19 FLO: Do I put it ↑under:::r?
 20 (0.4)
 21 TEO: Let's SEE what ↑happens (.) >try stuff out<
 22 FLO: *[((types fill(0,0,0) underneath last ellipse))*
 23 *[>Oh boy oh boy oh boy<*
 24 *((runs code and dog turns black))*

25 TEO: So what happened. (1.3)
 26 [Same thing?]
 27 FLO: [((wails nervously))]
 28 [uh: yes]
 29 NIC: [That's scary:: (.) what?]
 30 DAV: Wait did it do the same thing? (.) Flo (.) or did it do something
 31 [different when you put it belo::w]
 32 FLO: [It (.) did (.)] the same thing.
 33 **((runs the code repeatedly and dog repeatedly fills in black))**
 34 ALL: ((light laughter))
 35 TEO: hm: hm::: how should we tackle this (0.4) WHERE else are you using fill
 36 in your code
 37 FLO: **((scrolls up to the top of her code))**
 38 NOWhere else
 39 TEO: Hm:: that's interesting (0.7) what do you think about tha::t
 40 FLO: **((scrolls down and clicks next to fill function))**
 41 ((light wail)) Oh my GO::D
 42 TEO: ((light laughter)) (1.0) I know (.) I'm just trying to make you struggle
 43 a little bit (laughter)
 44 FLO: Aww::: [(no way (.) no way)]
 45 TEO: [((laughs)]
 46 It's IN good spirits::
 47 FLO: I thought you were my frie:::nd.
 48 TEO: ((laughs)) FRIENDS can let people- let their friends struggle,
 49 NIC: No ((laughs))
 50 FLO: **((makes an exaggerated horrified face at Teo))** What if they're gonna
 51 die::: and they're like [help me
 52 TEO: [Are you gonna die?
 53 I wouldn't let you just die::: Flo::o ((laughs))

This PixelBots challenge came with starter code. See the starter code below. Sal had not changed any of this code at this point in the transcript.

```
1 // In this challenge, you only
2 // need to define the function.
3
4
5 // move to lower-right square
6 // (lower-left corner)
7 turnRight()
8 forward(5)
9
10 blackSquare()
11
12
13 // move to upper-right square
14 // (lower-right corner)
15 turnLeft()
16 forward(4)
17
18 blackSquare()
19
20
21 // move to left square
22 // (upper-left corner)
23 turnLeft()
24 forward(4)
25 turnLeft()
26 forward(2)
27
```

Panda's starting location and target image (grey squares need to be painted black)

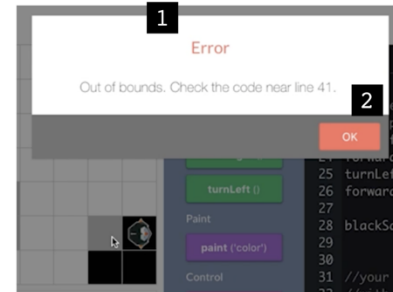


```
01 ((SAL runs code))
02 1((out of bounds error pops up))
03 SAL: No[::pe
04 INS: [>Alright now: what type of error do you have<
05 (0.6)
06 SAL: Now:: (.) out of bounds (.)
07 check [(the)
08 INS: [>Out of< boun::ds right,
09 (0.2)
10 SAL: Yeah
11 (3.2)
12 SAL: 2((clicks OK))
13 oTurn lefto
14 (1.2)
```

Sal had been working to define blackSquare(). Here is his function definition at the start of the transcript.

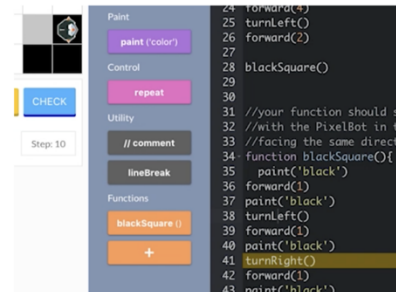
```
31 //your function should start and stop
32 //with the PixelBot in the same place
33 //facing the same direction
34 function blackSquare(){
35   paint('black')
36   forward(1)
37   paint('black')
38   turnLeft()
39   forward(1)
40   paint('black')
41   turnRight()
42   forward(1)
43   paint('black')
44
45 }
```

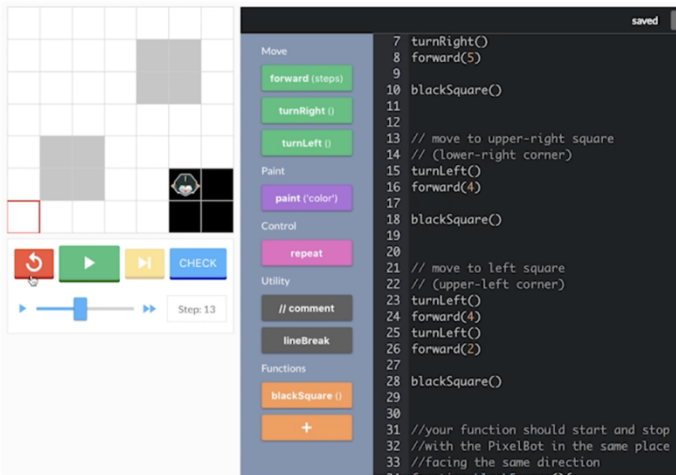
```
15 INS: What (.) tool should you use it when you want to debug
16 (0.6)
17 SAL: Stepper [(0.6) ] owaito
18 [((clicks play))]
19 (2.6)
20 INS: [You can always just hit the red button, hit the red
21 button, (0.6) and then you can always just start (like looking at your code)
22 SAL: [((clicks OK on error message again, clicks the red reset button, and then starts
23 clicking the yellow stepper button))
24 (3.6)
25 INS: You should look at- you should (.) be
26 looking at your code (0.8) to
27 [see owhat happenso
28 SAL: [3((scrolls up in text editor))
29 Oh (.) kay
30 INS: 4So now you're in your function black
31 square right?
32 SAL: [Yeah (4.3) oBlacko (turn)
33 [((steps through to turnLeft on line 38))
34 INS: What's going to happen next?
35 (0.4)
36 SAL: Then move [f::orward?
37 5[((clicks stepper))
38 (0.4)
39 INS: Then (watch)
40 (0.6)
41 SAL: And it paints [bla:ck
42 [((clicks stepper))
43 (0.6)
44 INS: Then what
45 (1.0)
46 SAL: [Turn ri:ght,
47 6[((clicks stepper))
48 ((changes turnRight to
49 turnLeft and clicks play))
```



```
14 // (lower-right corner)
15 turnLeft()
16 forward(4)
17
18 blackSquare()
19
20
21 // move to left square
22 // (upper-left corner)
23 turnLeft()
24 forward(4)
25 turnLeft()
26 forward(2)
27
28 blackSquare()
29
30
31 //your function should start and stop
32 //with the PixelBot in the same place
33 //facing the same direction
34 function blackSquare(){
35   paint('black')
36   forward(1)
37   paint('black')
38   turnLeft()
39   forward(1)
40   paint('black')
41   turnRight()
42   forward(1)
43   paint('black')
44
```

```
34 function blackSquare(){
35   paint('black')
36   forward(1)
37   paint('black')
38   turnLeft()
39   forward(1)
40   paint('black')
41   turnRight()
42   forward(1)
43   paint('black')
44
```





1

((SAL plays code; code throws an out-of-bounds error; student clicks OK on error message; after clicking OK, 1 the laptop screen is in this state))

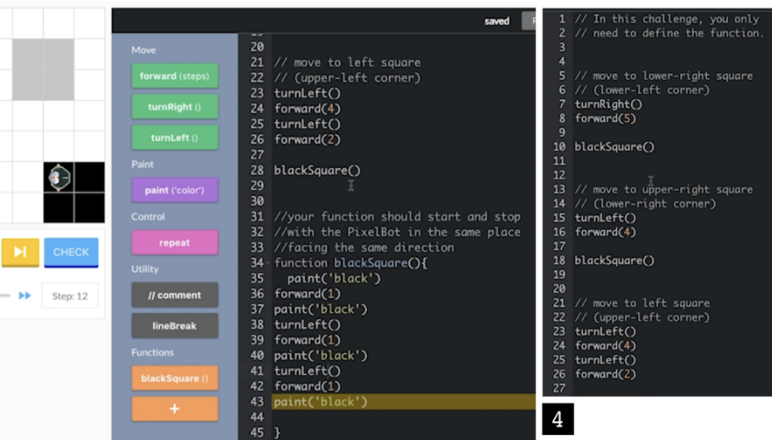
01 INS: Uh:: okay so let's (.) use the stepper tool again (.)
 02 but let's start off uh- at the top (.) scroll all the
 03 way up on your otext editor o [so we can see oall the
 04 SAL: 2 [((scrolls up))
 05 INS: code (.) We can see what- what's happening o (.) so
 06 what's the first thing it's going to do
 07 SAL: [So it's gonna (.) turn ri::ght?
 08 [((resets code with red button))
 09 ((clicks stepper button and line highlight covers
 10 turnRight() on line 7))

[Section removed from transcript. Student and instructor go back and forth with the instructor asking the student to predict what line of code will be read next as they step through the code]

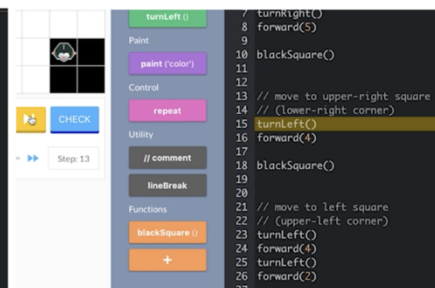
11 INS: Okay (0.4) Then what (.) 3 What is it gonna do ne:xt
 12 [(1.4) oafter () o
 13 SAL: [4 ((scrolls up))
 14 (3.2)
 15 SAL: 5 ((clicks stepper)) Turn le::ft
 16 INS: Uh huh (1.2) >Is that what you wanted<
 17 SAL: [Nope] so::
 18 [((shakes head))]

```
1 // In this challenge, you only
2 // need to define the function.
3
4
5 // move to lower-right square
6 // (lower-left corner)
7 turnRight()
8 forward(5)
9
10 blackSquare()
11
12
13 // move to upper-right square
14 // (lower-right corner)
15 turnLeft()
16 forward(4)
17
18 blackSquare()
19
20
21 // move to left square
22 // (upper-left corner)
23 turnLeft()
24 forward(4)
25 turnLeft()
26 forward(2)
27
```

2

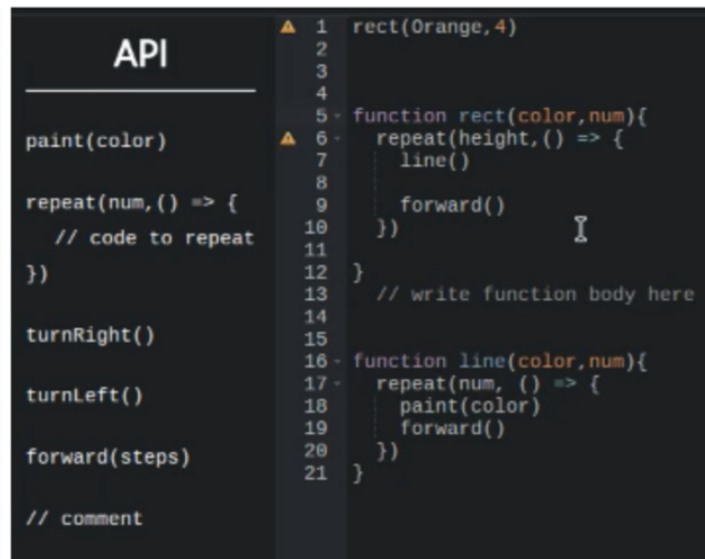
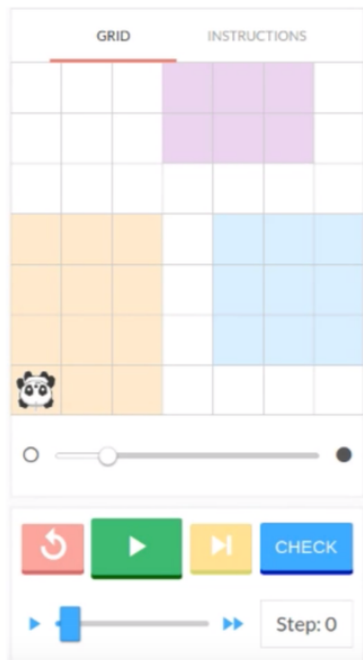


3



5

4



```

01 IVAN: [ⓂIm ↓stuckⓂ]
02      [(looks up at Jad, who is standing next to him)]
03      (1.0)
04 JAD: >You're stuck?<
05 IVAN: [↓Uh::u:::~:]
06      [(tilts head up with eyes pointed toward ceiling)]
07 JAD: [(taps Ivan's shoulder twice)]

08 IVAN: [↓uh::h]
09 JAD: [Think about it.]
10      [I think you know what the ( ) is]
11      [(leans against the back wall of the classroom,
12      glancing at Ivan's screen/code briefly)]
13 IVAN: I'm=
14 JAD: =(But) you're not thinking hard enough.
15 IVAN: I'm not THINnking
16 JAD: ( ) why aren't you thinking
17      (1.2)
18 IVAN: [ⓂI'm not (.) meant for co::dingⓂ]
19      [(Ivan shakes his head back and forth,
20      then faces down, looking into his lap)]
21 JAD: No you ar::re you're really good at coding.
22      Every time I leave I feel like you: (.) are able to do it
23      [(Ivan sticks out his lower lip in what resembles a pout)]
24 IVAN: EVERYtime I get STUCK in the ↓code (1.3) I got >so many< bu::gs.
25      (0.4)
26      Look- two bu:gs
27      (0.8)
28 JAD: >But you know how to DEbug it now.<
29      (1.4)
30 IVAN: [Nah:: I just click and {I'm just} wondering what happened with]
31      [(Ivan moves his cursor over to the error message on line 6)] ]

```


01 DEX: [oh::: my goodness (.) stop copying ((light laugh))
 02 [*((smiling widely))*
 03 CAM: [*((smiles widely))*
 04 [why:::
 05 DEX: [*((sits behind Cam and reaches over his shoulder to take over*
 06 *his keyboard; continues smiling))*
 07 [no:]
 08 CAM: [I want] to do (it)
 09 [*((continues smiling widely))*
 10 DEX: no- no-no-no-no-no-no (.) no copying
 11 CAM: hey I already did=
 12 DEX: [=you gotta do it on your own
 13 [*((starts deleting poké ball code in Cam's program))*

```
1  function setup() {
2      createCanvas(windowWidth, windowHeight);
3      background(100);
4
5  }
6
7  function draw() {
8      ellipse(80, 80, 20, 20)
9      rect(160, 160, 20, 20)
10
11 }
```

```
12 function preload() {
13     sound1 = loadSound('foundPokeball.wav')
14 }
15 function keyPressed() {
16     if(key == "c" || key == 'C') {
17         randomPokeball()
18         sound5.play()
19     }
20 }
```

```
mySketch, Line 14: randomPokeball is not defined
mySketch, Line 14: randomPokeball is not defined
mySketch, Line 14: randomPokeball is not defined
mySketch, Line 14: randomPokeball is not defined
mySketch, Line 14: randomPokeball is not defined
mySketch, Line 14: randomPokeball is not defined
```

14 CAM: kay
 15 [*((frowns and then motions towards his face with one finger as if a*
 16 *tear was running down his left cheek))*
 17 DEX: >sadly<
 18 [*((deletes additional poké ball code in Cam's program))*
 19 CAM: °sa::dly°
 20 DEX: [okay? so you made a rectangle and an ellipse (.) now
 21 [*((clicks in part of program where Cam can start working on*
 22 *triangle))*
 23 CAM: °yeah°
 24 DEX: try to do a triangle (shape)
 25 CAM: kay
 26 (1.2)
 27 DEX: >know how to do a triangle?< (.)
 28 CAM: ya (.)
 29 DEX: >go for a triangle.<
 30 (0.6)
 31 CAM: [((long, pronounced exhale))
 32 [*((starts typing triangle function into code editor))*
 33 sa:d

```
7  function draw() {
8      ellipse(80, 80, 20, 20)
9      rect(160, 160, 20, 20)
10     tri
11 }
```